



Guide des IHM d'administration

Principes, style et mécanismes réutilisables
issus de l'administration du valorisateur

Référence d'implémentation : back-office Quarkus / Qute du moteur de valorisation Intermarché
Destiné aux équipes construisant d'autres écrans d'administration

1. Intention

Ce document décrit les conventions d'une IHM d'administration existante, afin que d'autres back-offices puissent être construits avec la même apparence et les mêmes réflexes. Il ne s'agit pas d'un cadre logiciel à installer : c'est un ensemble de décisions à reproduire, dont la plupart tiennent en quelques lignes de CSS et un patron de code.

Le public visé est l'outil interne : un opérateur qui manipule des données de référence toute la journée. La priorité va donc à la **densité d'information**, à la **prévisibilité** et à la **réversibilité**, pas à l'effet visuel. Un écran d'administration réussi est un écran qu'on cesse de regarder.

Principe directeur. Toute décision d'interface se tranche en faveur de l'opérateur qui connaît déjà l'outil, pas du visiteur qui le découvre. Les raccourcis et la compacité l'emportent sur la pédagogie ; l'explication passe par l'infobulle et le texte d'aide, jamais par l'encombrement de l'écran.

2. Jetons visuels

Toute la charte tient dans une dizaine de variables CSS déclarées sur `:root`. Les reprendre telles quelles suffit à rendre un nouveau back-office visuellement solidaire de celui-ci. Aucune couleur ne doit être écrite en dur ailleurs.



`--im-red`
Identité, action principale,
en-tête de section



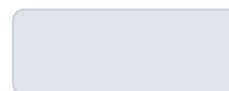
`--im-red-dark`
État survolé du rouge,
texte d'alerte



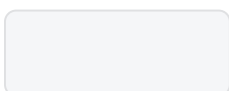
`--im-ink`
Texte principal



`--im-ink-soft`
Texte secondaire, libellés
de colonnes



`--im-line`
Filets, bordures,
séparateurs



`--im-bg`
Fond de page, zones en
retrait



`--im-surface`
Cartes, en-tête, surfaces
actives



`--im-accent`
Succès, validation, cible
de dépôt



`--im-warn`
Avertissement, erreur de
saisie

```

:root {
  --im-red: #e2001a;      --im-red-dark: #b40015;
  --im-ink: #16181d;      --im-ink-soft: #5b6270;
  --im-line: #dfe3ea;     --im-bg: #f5f6f8;
  --im-surface: #ffffff;  --im-accent: #0a7d5a;
  --im-warn: #b06000;
  --radius: 8px;
  --shadow: 0 1px 2px rgba(16,18,23,.06), 0 4px 16px rgba(16,18,23,.05);
}

```

Typographie et rythme

Une seule pile de polices système, une seule taille de base. La hiérarchie se fait par la graisse et la couleur, pas par la multiplication des tailles.

```

body {
  font: 15px/1.5 -apple-system, BlinkMacSystemFont, "Segoe UI",
    Roboto, Helvetica, Arial, sans-serif;
  background: var(--im-bg);
  color: var(--im-ink);
}

```

- Corps de texte et champs de saisie : 15 px. Les tableaux descendent à 13 px.
- Les en-têtes de colonnes et micro-libellés : 11 px, majuscules, interlettrage `.04em`, couleur `--im-ink-soft`.
- Rayon d'angle unique : `--radius` (8 px) pour les cartes, 5 à 7 px pour les contrôles.
- Une seule ombre, `--shadow`, réservée aux cartes. Pas d'ombre sur les contrôles.

3. Squelette de page

Toutes les pages passent par un gabarit unique : en-tête applicatif, zone principale, pied de page. Un écran n'écrit jamais son propre en-tête.

```
<header class="app-header">  marque + navigation + identité connectée
<main class="app-main">      contenu de l'écran
<footer class="app-footer">  mention de contexte
```

Organisation de la navigation

La barre de navigation est **plate et courte** : une entrée par grand objet métier, aucun sous-menu, aucun menu déroulant. Sur la référence, quatre entrées suffisent à couvrir l'application. C'est une contrainte volontaire : si une cinquième ou sixième entrée devient nécessaire, c'est le découpage fonctionnel qu'il faut réexaminer, pas la barre qu'il faut hiérarchiser.

```
[marque]  Offers  Store groups  Valuations  Users*  [identité]

* visible pour les seuls administrateurs
```

Trois règles portent cette organisation.

- **Une entrée = un objet métier au pluriel**, jamais un verbe ni une action. Les actions vivent dans l'écran de l'objet, pas dans le menu. On navigue vers « Offres », puis on y crée une offre.
- **L'ordre suit la fréquence d'usage**, du plus courant au plus rare, et non l'ordre alphabétique ou celui du modèle de données. L'administration des utilisateurs, consultée épisodiquement, est en dernier.
- **Les entrées réservées sont conditionnées par le rôle** dans le gabarit, et la ressource cible porte de son côté une annotation de rôle. Masquer un lien n'est pas une protection : c'est un confort d'affichage doublé d'un contrôle réel côté serveur.

La marque à gauche est aussi le lien de retour vers l'écran d'accueil — l'entrée la plus fréquemment utilisée sert de point de repli. À droite, le bloc d'identité regroupe le nom connecté, l'accès au changement de mot de passe et la déconnexion ; il n'apparaît qu'une fois authentifié.

Pas de fil d'Ariane, pas de deuxième niveau. Un back-office dont la navigation tient sur une ligne n'a pas besoin de fil d'Ariane : la profondeur maximale est de deux écrans (liste, puis détail ou formulaire), et le retour se fait par une commande explicite dans la coiffe de l'écran (« Back to list »), pas par une reconstruction de chemin.

Coiffe d'écran

Chaque écran s'ouvre sur le même bloc : titre, sous-titre explicatif, actions à droite.

```
<div class="page-head">
  <div>
    <h1>Store groups</h1>
    <p class="page-sub">{view.summary}</p>
  </div>
  <div class="page-actions">...</div>
</div>
```

Le sous-titre n'est pas décoratif : il porte l'état courant — nombre d'éléments, filtre appliqué, portée de la sélection. C'est là que l'opérateur vérifie qu'il regarde bien ce qu'il croit.

4. Vocabulaire de composants

Un jeu restreint de classes couvre la quasi-totalité des besoins. Les réutiliser garantit l'homogénéité ; en créer de nouvelles doit rester exceptionnel.

CLASSE	RÔLE
<code>.card</code>	Bloc de contenu sur fond blanc, coins arrondis, ombre douce. Unité de découpage de l'écran.
<code>.card-title</code> / <code>.card-title-row</code>	Titre de bloc, éventuellement avec un contrôle aligné à droite.
<code>.data-table</code>	Tableau de données : en-têtes discrets, filets fins, colonnes d'action à droite.
<code>.btn</code> + <code>.btn-primary</code> / <code>.btn-ghost</code> / <code>.btn-danger</code> / <code>.btn-small</code>	Une seule action principale par écran ; le reste en <i>ghost</i> . Le rouge <i>danger</i> est réservé à la destruction.
<code>.badge</code> + <code>.badge-ok</code> / <code>.badge-off</code> / <code>.badge-error</code>	État en un coup d'œil dans une cellule de tableau.
<code>.alert</code> + <code>.alert-ok</code> / <code>.alert-error</code>	Message de résultat après une action.
<code>.chip</code> / <code>.ac-chip</code>	Valeur sélectionnée dans une saisie multiple, avec sa croix de retrait.
<code>.field</code> / <code>.field-grid</code> / <code>.field-label</code> / <code>.field-hint</code>	Unité de formulaire et sa grille responsive.
<code>.input-control</code> / <code>.input-unit</code> / <code>.input-suffix</code>	Champ de saisie, éventuellement suffixé d'une unité (€, %, kg).
<code>.filter-bar</code> / <code>.pager</code>	Barre de filtres et pagination, en haut et en bas des listes.
<code>.empty-state</code> / <code>.placeholder-note</code>	Absence de données, expliquée plutôt que laissée vide.
<code>.is-hidden</code>	Seul utilitaire d'affichage. Pas de bibliothèque de classes utilitaires.

Découpage des feuilles de style. Un fichier `base.css` porte les jetons et tous les composants partagés ; un fichier par domaine fonctionnel porte le reste. Un composant utilisé par deux écrans remonte dans `base.css` — il ne se duplique jamais. La duplication d'une règle entre deux feuilles est une dette : lors d'une correction, on ne sait plus laquelle fait effet.

5. Mécanismes transposables

Au-delà du style, quatre mécanismes portent l'essentiel de la valeur et méritent d'être repris tels quels.

5.1 Vue de liste générique

Un seul objet de présentation, paramétré, sert toutes les listes : filtres, tri, pagination, message ponctuel et droit d'écriture. Il ne dépend d'aucun type métier et se construit en fin de traitement, une fois la requête exécutée.

```
ListView<T>(rows, basePath, filters, sortKey, descending,
            currentPage, pageCount, totalCount, pageSize,
            itemLabel, notice, noticeOk, canWrite)
```

Règle d'or. L'état de la liste vit *entièrement dans l'URL* : filtres, clé de tri, sens et numéro de page. Rien en session, rien en mémoire de composant. Une liste filtrée est donc partageable par simple copie du lien, rechargeable à l'identique, et le retour arrière du navigateur fonctionne sans code.

FABRICATION DES LIENS

Le gabarit ne construit jamais de chaîne de requête. Une méthode privée reconstitue l'URL de base à partir des filtres non vides, du tri et du sens ; les méthodes publiques n'ajoutent que ce qui les concerne.

```
baseUrl() = basePath
          + chaque filtre non vide
          + sort + dir

pageUrl(n)      = baseUrl() + page=n
sortUrl(colonne) = baseUrl() avec sort/dir remplacés
```

C'est ce qui fait que **les filtres survivent au tri et à la pagination** : ils sont réinjectés dans chaque lien produit. Un filtre vide ou blanc est omis, pour ne pas encombrer l'URL de paramètres inutiles.

TRI : BASCULE ET INDICATEUR

Un en-tête de colonne triable est un simple lien. Le sens s'inverse uniquement quand on reclique sur la colonne *déjà* triée ; changer de colonne repart en ordre croissant.

```
reverse = (colonne == tri courant) && !descendant
dir      = reverse ? "desc" : "asc"
```

L'indicateur visuel suit la même logique : chaîne vide si la colonne n'est pas celle du tri, sinon un chevron bas (▼) ou haut (▲). Dans le gabarit :

```
<a class="col-sort" href="{view.sortUrl('createdAt')}}">
  When <span class="sort-arrow">{view.sortIndicator('createdAt')}</span>
</a>
```

TRI : RESTRICTION CÔTÉ SERVEUR

La clé de tri arrive de l'URL, donc de l'extérieur. Elle est confrontée à une liste blanche déclarée en constantes, et toute valeur inconnue retombe sur le tri par défaut. C'est à la fois une protection contre l'injection dans la clause de tri et une garantie de robustesse face à un lien périmé.

```
private static final Set<String> SORTABLE =
    Set.of(SORT_DATE, SORT_STORE, SORT_DURATION, SORT_TOTAL);

String sortKey = SORTABLE.contains(sort) ? sort : SORT_DATE;
```

PAGINATION : BORNAGE SYSTÉMATIQUE

Le numéro de page arrive lui aussi de l'URL. Il est ramené dans les bornes réelles plutôt que de produire une page vide ou une erreur — un lien vers la page 40 d'une liste qui n'en compte plus que 3 affiche la page 3.

```
int pageCount = Math.max(1, query.pageCount());
int currentPage = Math.min(Math.max(page, 1), pageCount);
```

La taille de page est une constante par écran (25 sur la référence), jamais un paramètre d'URL : elle n'est pas à la main de l'appelant.

PAGINATION : RENDU

Le bloc de pagination n'apparaît que si la liste est effectivement paginée. Les quatre commandes — première, précédente, suivante, dernière — sont toujours présentes : désactivées visuellement plutôt que retirées, pour que la barre ne se réorganise pas d'une page à l'autre.

```
Page 2 of 7      Showing 26–50 of 168
```

Le libellé de page indique la position réelle des lignes affichées, pas seulement le numéro de page. Il est calculé à partir du nombre de lignes réellement rendues, donc reste juste sur la dernière page incomplète.

BARRE DE FILTRES

La barre de filtres est un formulaire en `GET` — c'est ce qui fait naturellement remonter les filtres dans l'URL, sans code. Deux points de méthode :

- Le tri courant est transporté en champs cachés, sinon filtrer réinitialiserait le tri.
- Chaque contrôle est réalimenté par `view.filter('nom')`, pour que la barre reflète l'état après rechargement.
- Le bouton de réinitialisation n'apparaît que si un filtre est actif (`view.filtered`) : un lien nu vers le chemin de base.

```
<form class="filter-bar" method="get" action="/ui/objets">
  <input type="hidden" name="sort" value="{view.sort}">
  <input type="hidden" name="dir" value="{view.direction}">
  ...champs réalimentés par view.filter('...')...
  <button class="btn btn-primary">Filter</button>
  {#if view.filtered}<a class="btn btn-ghost" href="/ui/objets">Reset</a>{/if}
</form>
```

Noter que la pagination n'est pas transportée en champ caché : filtrer doit ramener à la première page, ce qu'on obtient en ne transmettant pas le numéro courant.

5.2 Formulaire piloté par schéma

Les écrans de configuration ne codent pas leurs champs : le serveur publie un schéma JSON par type d'objet, et un script unique en dérive le formulaire. Ajouter un type de configuration ne demande aucune ligne d'IHM.

Le schéma standard est enrichi d'annotations d'affichage, ignorées par les validateurs :

ANNOTATION

EFFET

x-widget	Choisit le contrôle : <code>money</code> , <code>rate</code> , <code>percent</code> , <code>quantity</code> , <code>ean</code> , <code>store-code</code> , <code>object-list</code> ...
x-label	Libellé affiché, à la place du nom technique.
x-item-label	Nom de l'élément dans une liste répétable (« ligne », « palier »).
x-unit-from	Unité dérivée d'un autre champ du même objet.

Le même schéma sert à la validation côté serveur et à la génération côté client : une seule source de vérité, donc pas de divergence possible entre ce que le formulaire accepte et ce que le service exige. Une bascule *Form / JSON* laisse toujours l'accès à la donnée brute — indispensable pour les cas que le formulaire ne couvre pas encore.

Piège vécu. Le widget d'un taux doit distinguer la donnée *stockée en fraction* (0,20 affiché « 20 % ») de celle *déjà exprimée en pourcentage* (50 pour « 50 % »). Un widget unique pour les deux produit des erreurs d'un facteur cent, invisibles à la relecture. Deux widgets distincts, deux sémantiques.

5.3 Action, redirection, message

Toute action modifiante suit le même cycle : soumission en POST, puis redirection vers la liste avec un message en paramètre. Jamais de rendu direct en réponse à un POST.

```
POST /ui/objets/{id}/action
→ 303 See Other → /ui/objets?notice=...&noticeOk=true|false
```

Le rechargement de la page ne rejoue donc jamais l'action, et le message s'affiche une fois puis disparaît de la navigation. Le même canal sert aux succès et aux échecs, distingués par un seul booléen.

5.4 Édition directe des structures

Pour les objets dont la valeur est dans la *relation* — hiérarchies, appartenances, regroupements — le formulaire champ par champ est un mauvais outil. L'atelier de manipulation directe (glisser-déposer, sélection multiple, repli des branches) donne un résultat très supérieur.

Le patron : l'écran charge un modèle JSON complet, le manipule intégralement côté client, et **soumet la structure entière** en une fois. Le serveur remplace l'état existant par celui reçu.

Deux conséquences à assumer. La suppression se fait par omission — un élément absent de la soumission est supprimé — ce qui exige que l'écran soit la source complète de vérité, sans édition concurrente. Et surtout : *toute action destructive doit avoir une commande visible*. Une suppression accessible uniquement par la touche `Suppr` est une fonction que personne ne trouve.

6. Règles de comportement

Destruction et réversibilité

- Une commande de suppression est **visible** : icône explicite sur la ligne, révélée au survol, toujours présente au focus clavier. Le raccourci clavier vient en complément, jamais seul.
- La confirmation est **proportionnée** : rien pour un élément vide, une question explicite pour un élément qui porte du contenu — en énonçant ce qu'il contient et ce qu'il adviendra de ce contenu.
- Quand la modification n'est effective qu'après enregistrement, la confirmation peut être allégée : l'abandon reste possible.

États et vides

- Une liste vide s'explique : pourquoi il n'y a rien, et quelle action changerait cela. Jamais un tableau nu.
- Un chargement long affiche un état intermédiaire ; un échec de rafraîchissement conserve les données précédentes plutôt que de vider l'écran.
- Une valeur absente s'écrit par un tiret cadratin, jamais par une cellule vide qu'on pourrait confondre avec un zéro.

Droits

- Le droit d'écriture est porté par la vue (`canWrite`) et conditionne l'affichage des commandes d'édition.
- Il est **toujours doublé côté serveur** par une annotation de rôle sur la ressource. L'absence de bouton n'est pas une protection.

Rafraîchissement

- Un écran de supervision peut se rafraîchir seul, mais **uniquement la zone de données** — jamais la page entière, qui détruirait la saisie en cours.
- Le rafraîchissement transmet les filtres courants, se met en pause quand l'onglet est en arrière-plan, et n'empile pas les requêtes.

7. Erreurs à ne pas reproduire

Ces points ont tous coûté du temps sur l'implémentation de référence. Les connaître fait gagner ce temps.

SYMPTÔME	CAUSE ET PARADE
Entité HTML affichée littéralement (<code>&mdash;</code>)	Le moteur de gabarit échappe le résultat des <i>expressions</i> , pas le texte statique. Utiliser le caractère réel, ou la directive d'échappement brut.
Formulaire systématiquement vide malgré des données présentes	JSON injecté dans une balise <code><script></code> sans directive brute : les guillemets échappés cassent l'analyse. Toujours <code>.raw</code> pour du JSON.
Suffixe d'unité chevauchant les flèches d'un champ numérique	Le suffixe est positionné là où le navigateur dessine ses propres commandes. Le décaler et augmenter le retrait interne.
Tableau débordant de sa carte	Deux colonnes côte à côte pour un contenu qui n'y tient pas. Empiler verticalement, fixer la répartition des colonnes, autoriser le retour à la ligne.

Règles CSS dupliquées dans deux feuilles	On corrige le fichier qui n'est pas chargé. Une seule définition, dans la feuille de base.
Feuilles et gabarits jamais référencés	Vestiges d'écrans remplacés. Les supprimer : ils font perdre du temps à chaque intervention.

8. Liste de contrôle

Avant de considérer qu'un nouvel écran d'administration est homogène avec l'existant :

- ☐ Les jetons CSS sont importés tels quels ; aucune couleur n'est écrite en dur.
- ☐ L'écran passe par le gabarit commun et n'écrit pas son propre en-tête.
- ☐ La coiffe porte un titre et un sous-titre d'état.
- ☐ Les listes utilisent la vue générique ; filtres, tri et page sont entièrement dans l'URL.
- ☐ Les clés de tri sont restreintes par liste blanche côté serveur ; le numéro de page est borné.
- ☐ La barre de filtres est un formulaire GET qui transporte le tri et abandonne la page.
- ☐ La navigation est plate, une entrée par objet métier, ordonnée par fréquence d'usage.
- ☐ Une seule action principale par écran ; le reste en secondaire.
- ☐ Toute action modifiante suit le cycle POST → redirection → message.
- ☐ Les actions destructives ont une commande visible et une confirmation proportionnée.
- ☐ Le droit d'écriture est vérifié côté serveur, pas seulement masqué dans l'IHM.
- ☐ Les états vides sont expliqués.
- ☐ Aucune feuille de style ni gabarit orphelin n'est laissé dans le dépôt.